

程序设计实习 · 汇报与代码答辩

植被指数智能分析平台

源代码清单、目录设计、模块划分、接口与分层实现主手册

项目：植被指数提取算法封装与 Web 服务实现

技术栈：FastAPI + Rasterio + NumPy/Joblib/PyTorch + Vue 3 + pygeoapi

生成日期：2026-07-02

代码基线：当前工作区实现，35 个内置植被指数

用途：源码导读、接口演示、架构说明、分块计算讲解与答辩备忘

阅读导航

建议汇报顺序：先讲第 2、4、8、9、10、12 章，再演示第 7 章接口，最后用第 18 章答辩问答收尾。

- 1. 文档定位与汇报速览
- 2. 项目目标、需求与已实现范围
- 3. 总体架构与端到端数据流
- 4. 文件目录设计与模块划分
- 5. 全部源代码清单
- 6. 后端核心对象与依赖方向
- 7. 接口设计概要与接口文档查看方法
- 8. pygeoapi / OGC API - Processes 实现
- 9. 植被指数注册表设计
- 10. 多计算引擎设计
- 11. 自动引擎选择
- 12. Rasterio 分块计算完整设计
- 13. 资产、瓦片、统计和溯源产物
- 14. 异步任务、Celery、Redis 与取消
- 15. 智能分析 Agent 的实现与安全边界
- 16. Vue 前端各层实现
- 17. 部署、配置、测试与常见问题
- 18. 汇报演示脚本与高频答辩问答
- 附录 A. FastAPI 全接口表
- 附录 B. 35 个植被指数表

1. 文档定位与汇报速览

本手册不是需求规划书，而是对当前仓库真实代码的系统说明。它回答五个核心问题：代码放在哪里、模块为什么这样划分、请求如何进入计算、35 个指数如何共享同一套公式、大影像为什么不会一次性撑爆内存。

30 秒总述：平台以统一指数注册表为算法事实源，FastAPI/pygeoapi 把算法暴露为 OGC 风格服务，RasterPipeline 负责窗口化 I/O，NumPy/Joblib/PyTorch 负责数组计算，Celery 负责异步调度，Vue 3 工作台完成上传、波段映射、任务、地图、统计和 Agent 交互。

1.1 汇报时必须主动说明的边界

- 当前代码内置 35 个指数：30 个任务书核心指数，加 5 个旧服务兼容指数。
- 公式只定义一次，三种引擎共享 expression；不存在三份公式复制。
- 分块计算按窗口读写，内存占用主要与块大小、波段数和指数数相关，而不是与整幅影像面积线性绑定。
- 本地开发可使用线程池/eager；容器部署使用 Celery + Redis、MinIO、Nacos 和 Traefik。
- Agent 默认先生成计划，确认接口之后才真正提交计算。
- GPU、Docker、Nacos、MinIO 的真实运行效果取决于汇报机器环境；代码支持不等于当前机器已完成全部外部基础设施验证。

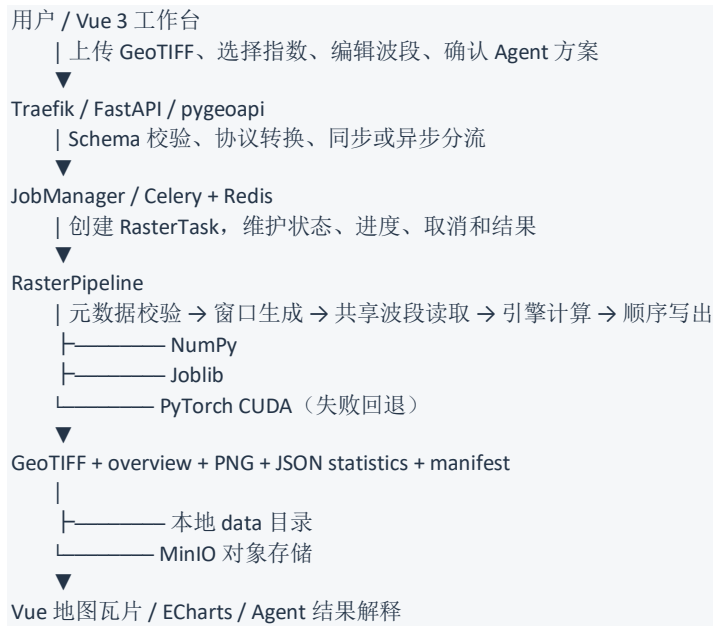
2. 项目目标、需求与已实现范围

任务书要求从算法层、服务层、调用层和部署层完成植被指数服务化。当前实现把这些要求拆成可测试的独立层，并通过 `/api/system/taskbook-coverage` 暴露覆盖清单。

任务书要求	当前实现	关键代码	说明
至少 30 种指数	35 个内置指数	backend/app/core/indices.py	统一注册、统一元数据

任务书要求	当前实现	关键代码	说明
Rasterio 分块	按 blockSize 生成 Window	services/raster_pipeline.py	共享波段读取
同步/异步调用	Prefer: respond-async	api/routes.py + services/jobs.py	同一任务模型
Celery + Redis	五级优先队列	celery_app.py + worker_tasks.py	部署模式启用
MinIO	输入/结果对象上传	services/assets.py	本地不可用时保留本地路径
pygeoapi	动态 Processor	pygeoapi_processor.py	单类处理全部指数
Nacos + Traefik	注册、心跳和配置桥	services/nacos.py + nacos_bridge.py	服务发现与网关解耦
Vue 3 前端	地图、上传、任务、Agent、统计	frontend/src/	Pinia 管理单一状态

3. 总体架构与端到端数据流



3.1 一次同步请求的完整调用链

1. 前端 `usePlatformApi.executeProcess` 向 `/processes/{id}/execution` 发送 JSON。

2. Pydantic `ExecutionRequest` 校验 source、indices、bands、engine、blockSize 和 priority。
3. 路由把请求转换成 `RasterTask`，并解析 MinIO objectKey 或本地路径。
4. `JobManager.execute\_sync` 直接调用 `RasterPipeline.run`。
5. 流水线读取影像元数据，计算必需逻辑波段并集，选择实际引擎。
6. 每个窗口只读取一次必需波段，三种引擎只接触 NumPy 数组/张量。
7. 每个指数写入独立 GeoTIFF，随后构建 overview、统计、预览与 manifest。
8. 响应返回产品路径、对象键、范围、CRS、统计、实际引擎和回退原因。

3.2 一次异步请求的区别

唯一关键区别是请求头包含 `Prefer: respond-async`。路由不直接运行流水线，而是调用 `JobManager.submit`。本地模式进入线程池，部署模式进入 Celery 队列。前端轮询 `/jobs/{id}`，成功后读取 `/jobs/{id}/results`。核心计算代码仍然只有一份。

4. 文件目录设计与模块划分

```

backend/app/
├─ api/          HTTP Schema 与路由编排
├─ core/         纯算法注册表
├─ engines/      数组计算后端
├─ services/     栅格、任务、资产、Agent、存储、Nacos
├─ main.py       FastAPI 装配
├─ worker_tasks.py Celery 入口
└─ pygeoapi_processor.py

frontend/src/
├─ components/   地图、Agent、资产、任务、统计与应用框架
├─ composables/  HTTP/SSE 与主题
├─ stores/       Pinia 单一工作台状态
├─ types/        前后端契约类型
├─ App.vue       根编排
└─ main.ts       启动入口

infra/
├─ pygeoapi/config.yml
└─ traefik/traefik.yml

```

4.1 划分原则

- 依赖方向自外向内：路由可以依赖服务，服务可以依赖核心；核心公式不反向依赖框架。

- 同一事实只保留一个来源：公式在注册表、前端状态在 Pinia、任务状态在 JobManager。
- 协议和执行分离：FastAPI 与 pygeoapi 只是两种入口，最终都进入 RasterPipeline。
- 计算和 I/O 分离：引擎只接收数组，Rasterio 只存在于流水线与分析服务。
- 部署和业务分离：Nacos、Traefik、Celery、MinIO 通过适配层接入，不污染公式。

5. 全部源代码清单

下表由生成脚本扫描当前仓库得到，包含全部自有 Python、Vue、TypeScript、CSS 源文件和关键部署配置。行数用于汇报定位，不是质量指标。

层	相对路径	行数	职责
后端入口/配置	backend/app/__init__.py	30	后端包初始化与运行环境兼容入口
接口层	backend/app/api/routes.py	764	平台 REST、OGC、Agent SSE 与瓦片路由
接口层	backend/app/api/schemas.py	168	HTTP 请求与 Agent 执行单模型
后端入口/配置	backend/app/celery_app.py	35	Celery 应用与优先队列配置
算法/引擎层	backend/app/core/indices.py	558	植被指数统一注册表与公式定义
算法/引擎层	backend/app/engines/base.py	45	计算引擎公共协议与输出清洗
算法/引擎层	backend/app/engines/joblib_engine.py	58	Joblib CPU 并行计算引擎
算法/引擎层	backend/app/engines/numpy_engine.py	41	NumPy 基线计算引擎
算法/引擎层	backend/app/engines/torch_engine.py	117	PyTorch CUDA 计算与回退引擎
后端入口/配置	backend/app/main.py	62	FastAPI 应用装配入口
后端入口/配置	backend/app/nacos_bridge.py	90	Nacos 到 Traefik 的动态路由同步桥
后端入口/配置	backend/app/pygeoapi_processor.py	86	pygeoapi 动态植被指数处理器
后端服务层	backend/app/services/advanced_analysis.py	181	自定义公式、变化检测和地块统计服务
后端服务层	backend/app/services/agent.py	519	需人工确认的植被分析 Agent
后端服务层	backend/app/services/agent_knowledge_store.py	216	Agent 外部知识内存/PostgreSQL 存储
后端服务层	backend/app/services/agent_session_store.py	157	Agent 会话与事件存储
后端服务层	backend/app/services/agent_tools.py	447	Agent 指数知识、动态注册和结果解释工具

层	相对路径	行数	职责
后端服务层	backend/app/services/assets.py	327	GeoTIFF 资产、overview、预览与 MinIO 适配
后端服务层	backend/app/services/custom_index_store.py	120	自定义指数 PostgreSQL 持久化
后端服务层	backend/app/services/jobs.py	213	本地线程池与 Celery 异步任务管理
后端服务层	backend/app/services/nacos.py	99	FastAPI 服务的 Nacos 注册与心跳
后端服务层	backend/app/services/planner.py	70	计算引擎自动选择器
后端服务层	backend/app/services/raster_pipeline.py	317	Rasterio 分块计算、统计、预览与溯源流水线
后端服务层	backend/app/services/tiles.py	221	GeoTIFF/COG 动态地图瓦片服务
后端入口/配置	backend/app/settings.py	43	应用配置模型
后端入口/配置	backend/app/worker_tasks.py	44	Celery Worker 栅格任务入口
后端入口/配置	backend/pyproject.toml	51	项目文件
后端入口/配置	backend/scripts/benchmark.py	62	多计算引擎公式微基准脚本
测试层	backend/tests/conftest.py	45	测试夹具与外部服务隔离
测试层	backend/tests/test_advanced_analysis.py	89	自定义公式、变化检测和区域统计测试
测试层	backend/tests/test_agent.py	171	Agent 推荐、确认、RAG 和知识隔离测试
测试层	backend/tests/test_api.py	341	FastAPI、OGC、上传、瓦片和 Agent 接口测试
测试层	backend/tests/test_assets.py	170	影像 overview 与波段元数据测试
测试层	backend/tests/test_indices.py	73	指数注册表、公式和多引擎一致性测试
测试层	backend/tests/test_raster_pipeline.py	53	分块流水线空间几何测试
部署层	compose.yml	191	部署或服务配置
项目配置	frontend/package.json	27	项目配置或类型配置
前端状态/类型层	frontend/src/App.vue	288	遥感工作台主壳组件

层	相对路径	行数	职责
前端状态/类型层	frontend/src/assets/main.css	150	应用全局设计变量与基础样式
前端组件层	frontend/src/components/AgentDrawer.vue	1937	智能体对话与执行单侧栏
前端组件层	frontend/src/components/AppStatusBar.vue	146	底部系统状态栏
前端组件层	frontend/src/components/AppToolBar.vue	276	应用顶部工具栏
前端组件层	frontend/src/components/AssetToolBar.vue	700	影像资产与波段映射工具栏
前端组件层	frontend/src/components/IndexCatalog.vue	491	植被指数目录与公式浏览器
前端组件层	frontend/src/components/JobProgressPanel.vue	400	异步任务进度与结果面板
前端组件层	frontend/src/components/MapWorkspace.vue	1343	MapLibre 遥感地图工作区
前端组件层	frontend/src/components/StatisticsDashboard.vue	230	结果统计与 ECharts 面板
前端状态/类型层	frontend/src/composables/usePlatformApi.ts	353	平台 HTTP 与 SSE 客户端
前端状态/类型层	frontend/src/composables/useTheme.ts	48	主题状态组合式函数
前端状态/类型层	frontend/src/main.ts	12	Vue 应用启动入口
前端状态/类型层	frontend/src/stores/workspace.ts	279	工作台 Pinia 单一状态源
前端状态/类型层	frontend/src/types/platform.ts	233	前后端共享业务类型
项目配置	frontend/vite.config.js	27	项目文件
部署层	infra/pygeoapi/config.yml	52	独立 pygeoapi 服务配置
部署层	infra/traefik/traefik.yml	18	部署或服务配置

6. 后端核心对象与依赖方向

对象	职责	输入	输出/状态	不得承担
IndexDefinition	指数契约	xp、bands、parameters	数组与公开元数据	Rasterio/HTTP
ExecutionRequest	HTTP 契约校验	JSON	强类型请求	文件计算
RasterTask	执行参数载体	解析后的路径与选项	可序列化任务	业务状态
ExecutionPlanner	引擎决策	尺寸、波段、指数、请求	selected/reason/memory	真实计算
ComputeEngine	窗口数组计算	定义列表、波段数组	EngineResult	文件 I/O
RasterPipeline	完整栅格执行	RasterTask	产品与 manifest	HTTP 状态码
JobManager	任务生命周期	RasterTask	JobRecord	公式实现
VegetationAgent	方案规划与解释	自然语言、波段、知识	待确认计划	未确认执行

7. 接口设计概要与接口文档查看方法

7.1 启动 FastAPI

```
cd backend
D:\miniconda\envs\giskeshe\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8011 --reload
```

7.2 浏览器入口

入口	地址	用途
Swagger UI	http://127.0.0.1:8011/docs	交互式查看 Schema 并直接 Try it out
ReDoc	http://127.0.0.1:8011/redoc	按模型和标签阅读接口说明
OpenAPI JSON	http://127.0.0.1:8011/openapi.json	供前端生成、测试或导入 Postman

入口	地址	用途
健康检查	http://127.0.0.1:8011/health	确认 FastAPI 进程已启动
指数目录	http://127.0.0.1:8011/api/indices	检查 35 个指数元数据
Process 目录	http://127.0.0.1:8011/processes	查看 OGC 风格处理能力

7.3 Swagger 演示建议

- 9. 先调用 `GET /health`，确认服务在线。
- 10. 调用 `GET /api/system/capabilities`，展示引擎、CUDA、指数数和存储模式。
- 11. 调用 `GET /api/indices/ndvi`，解释注册表元数据如何直接成为接口输出。
- 12. 调用 `POST /api/assets/inspect` 或上传接口，展示影像尺寸、CRS、波段和范围。
- 13. 调用 `POST /processes/ndvi/execution`，提交 source、bands、engine 和 blockSize。
- 14. 再用 `Prefer: respond-async` 演示返回 jobID，并查询 jobs/results。

7.4 典型执行请求

```
{
  "source": {"localPath": "data/uploads/example.tif"},
  "indices": ["ndvi"],
  "bands": {"red": 3, "nir": 4},
  "engine": "auto",
  "blockSize": 1024,
  "priority": 3,
  "preview": true,
  "statistics": true
}
```

接口设计重点：`bands` 使用逻辑波段名映射物理波段号，使公式不关心 Sentinel、Landsat 或无人机的实际波段顺序。

8. pygeoapi / OGC API - Processes 实现

项目同时保留 FastAPI 的 OGC 风格路由和独立 pygeoapi Processor。前者方便与现有平台、Agent、上传和任务中心统一；后者展示标准框架插件能力。两者最终复用相同的注册表和 RasterPipeline。

8.1 配置如何找到处理器

```
resources:
  spectral-index:
    type: process
    processor:
      name: app.pygeoapi_processor.SpectralIndexProcessor
```

`name` 是 Python 可导入路径。pygeoapi 实例化 `SpectralIndexProcessor`，构造函数把 `PROCESS\_METADATA` 交给 `BaseProcessor`；`execute(data)` 再解析 index/source/bands，构造 `RasterTask` 并返回 `(application/json, result)`。

8.2 为什么一个类能处理 35 个指数

Process 输入包含 `index` 字段。处理器通过 `get\_index(index\_id)` 从统一注册表获取定义，因此新增指数只需要扩展注册表，不需要新增 Processor 类、重复 YAML 或复制 I/O 逻辑。

8.3 如何查看 pygeoapi

15. 确保当前工作目录能导入 `backend/app`，并让 pygeoapi 使用 `infra/pygeoapi/config.yml`。
16. 开发模式可使用 `pygeoapi serve --config infra/pygeoapi/config.yml`，配置监听端口为 5000。
17. 访问 `http://localhost:5000/` 查看 Landing Page。
18. 访问 `http://localhost:5000/openapi.json` 获取 OpenAPI 文档；Landing Page 通常提供 Swagger UI/ReDoc 链接。
19. 访问 `http://localhost:5000/processes` 查看 Process 列表。
20. 访问 `/processes/spectral-index` 查看描述，POST `/processes/spectral-index/execution` 执行。
21. 异步任务按 OGC API - Processes 使用 `/jobs`、`/jobs/{jobId}` 和结果链接查看。

当前实现差异：独立 pygeoapi Processor 目前直接调用 RasterPipeline；平台级 FastAPI 路由拥有

更完整的上传、Agent、任务轮询、瓦片和扩展分析能力。答辩时应把它们表述为“标准框架入口”和“综合平台入口”。

9. 植被指数注册表设计

9.1 IndexDefinition 字段

字段	含义	为什么需要
id/name	机器编号与中文名	接口、文件名和 UI 统一定位
formula	人类可读公式	Swagger、前端公式卡和答辩说明
required_bands	逻辑波段集合	执行前验证与共享读取并集
expression	后端无关计算函数	同一公式运行在 NumPy 或 Torch
parameters	默认参数	SAVI/EVI 等参数化指数
expected_range	建议范围	UI 展示、异常识别和解释
categories/tags	分类与推荐标签	目录筛选和 Agent RAG
limitations	限制条件	避免把单一指数解释为确定诊断
amp_safe	混合精度安全标记	未来控制 GPU 半精度

9.2 公式零框架依赖

```
IndexDefinition(  
    id="ndvi",  
    required_bands=("nir", "red"),  
    expression=lambda xp, b, p:  
        safe_divide(xp, b["nir"] - b["red"], b["nir"] + b["red"]),  
)
```

`xp` 是数组 API：NumPy 引擎传 `numpy`，Torch 引擎传 `TorchArrayAPI`。公式不导入 Rasterio、FastAPI、Celery 或 MinIO，因此可以独立单元测试，也能新增 CuPy 等后端。

9.3 safe_divide 的意义

遥感指数分母可能接近 0。直接相除会产生无穷值并污染统计。`safe\_divide` 使用 `xp.where` 把绝对值小于 epsilon 的分母替换为 epsilon，使 NumPy 与 PyTorch 保持一致。随后 `sanitize\_result` 再把残余 NaN/Inf 转为统一 nodata。

9.4 新增指数的标准步骤

- 22. 在 `INDEX\_DEFINITIONS` 增加一个 `IndexDefinition`。
- 23. 使用逻辑波段名，不写物理波段号。
- 24. 所有除法使用 `safe\_divide`，平方根使用 `maximum(..., 0)` 防止负数。
- 25. 补齐 formula、description、range、categories、tags 和 limitations。
- 26. 在固定数组测试中验证数值有限，并验证 NumPy/Joblib/Torch 一致性。
- 27. 前端无需新增公式代码，重新加载 `/api/indices` 即可展示。

10. 多计算引擎设计

引擎	实现	适用场景	并行/设备	失败策略
NumPy	NumpyEngine	小影像、同步、正确性基线	CPU 顺序	直接抛出输入错误
Joblib	JoblibEngine	多指数、中大型 CPU	线程并行指数	缺少 joblib 时回退 NumPy
PyTorch	TorchEngine	大影像、多指数、CUDA	GPU inference_mode	导入/CUDA/运行失败回退 Joblib

10.1 为什么 Joblib 并行指数而不是并行写文件

同一窗口已在主线程读入内存。Joblib 并行执行多个指数公式，能够复用波段数组；计算完成后由 RasterPipeline 顺序写入不同输出文件。这样避免多个线程同时操作 Rasterio writer 的线程安全风险。

10.2 TorchArrayAPI 的作用

注册表达式需要 `abs/sqrt/sign/where/maximum` 等统一接口。PyTorch 对标量和张量的处理与 NumPy 略有差异，因此 `TorchArrayAPI` 做轻量适配，让公式本身无需出现 `torch.\*` 分支。

10.3 为什么 GPU 不一定更快

小窗口的主机到显存拷贝、张量构造和结果回传可能比公式计算更耗时。因此自动规划器只在像元规模大、指数数量多且 CUDA 可用时选择 Torch。实际阈值应由 `backend/scripts/benchmark.py` 在目标机器校准。

11. 自动引擎选择

`ExecutionPlanner.choose` 接收宽、高、波段数、指数数、请求引擎和是否同步，输出请求引擎、实际选择、原因和估算内存。显式指定 numpy/joblib/torch 时尊重用户选择；auto 才使用阈值。

- 同步或小于约 200 万像元：优先 NumPy，启动成本最低。
- 大于约 2000 万像元、至少 4 个指数且 CUDA 可用：选择 Torch。
- 其他中大型任务：选择 Joblib。
- 请求 Torch 但 CUDA 不可用：执行阶段由 TorchEngine 记录原因并回退。
- 响应和 manifest 同时记录 selectedEngine 与 actualEngine，避免把“计划使用”误当“实际使用”。

12. Rasterio 分块计算完整设计

一句话解释：把整幅影像切成许多小 Window；每次只读取当前窗口所需波段，计算全部指数后立即写回对应位置，因此峰值内存由块大小决定，而不是由整幅影像决定。

12.1 为什么需要分块

假设一幅 20000×20000、7 波段、float32 影像，整幅读入约需 10.4 GiB，另加多个指数输出和中间数组很容易超过内存。若块大小为 1024×1024，一个波段约 4 MiB，7 波段约 28 MiB；即使同时保留多个结果，内存仍处于可控范围。

12.2 窗口生成

```
for row in range(0, source.height, block_size):
    for column in range(0, source.width, block_size):
        Window(
            column,
            row,
            min(block_size, source.width - column),
            min(block_size, source.height - row),
        )
```

右边和下边最后一个窗口通常不足完整块，`min` 保证窗口不会越界。窗口的 row/column offset 保证写出时回到与源影像完全相同的像元位置。

12.3 必需波段并集与共享读取

先对所有指数的 `required\_bands` 求并集。例如 NDVI、EVI、GNDVI 同时执行时，只需读取 blue、green、red、nir 四个逻辑波段；不是每个指数分别读取。共享读取是多指数批处理最重要的 I/O 优化。

12.4 nodata 与 mask 处理

28. 使用 `source.read\_masks` 获取 Rasterio 有效性掩膜。
29. 若源影像声明 nodata，再用 `np.isclose` 合并显式 nodata。
30. 计算前把无效像元改为 NaN，让三种引擎遵循同一语义。
31. 全部必需波段掩膜做逻辑或：任一必需波段无效，该像元所有相关指数无效。
32. 写出前把无效像元设为固定 -9999，统计时排除该值。

12.5 计算块与 GeoTIFF 写块

请求 blockSize 被限制在 128~2048。计算窗口可以使用任意该范围整数；但 tiled GeoTIFF 的 blockxsize/blockysize 要满足 16 的倍数，因此不满足时写块回退 1024。计算块和存储块是两个概念：前者控制运行内存和进度，后者影响文件内部组织和读取性能。

12.6 顺序写出与空间信息保持

输出 profile 从源 profile 复制，只修改 driver、count、dtype、nodata、压缩和分块参数，因此 CRS、transform、width、height 保持不变。每个指数拥有独立 writer，当前窗口计算后写入相同 Window。

12.7 进度、取消和资源清理

窗口总数在执行前已知，进度为 ``current/len(windows)``。每个窗口前检查 ``is_cancelled``；无论成功、失败还是取消，``finally`` 都关闭所有 writer，防止文件句柄泄漏和损坏。

12.8 overview、统计、预览与 manifest

产物	生成方式	用途
GeoTIFF	逐窗口写入 float32, DEFLATE + predictor	精确结果与下载
内部 overview	2/4/8/16 average	地图缩放快速读取
PNG 预览	百分位拉伸与色带	列表和无瓦片时占位
statistics	有效像元 min/max/mean/std/histogram	图表和 Agent 解释
manifest.json	输入哈希、参数、引擎、回退、环境、产品	复现和审计
MinIO 对象	可用时上传输出和预览	跨服务访问

12.9 当前实现与进一步优化

- 当前实现先构造全部 Window 列表，超大影像可进一步改为生成器以降低列表开销。

- 当前统计阶段会读取整个单波段结果；超大结果可改为在线累计统计与分块直方图。
- 当前 Joblib 并行的是窗口内多个指数；未来可建立读取-计算-写入有界流水队列。
- Torch 当前按窗口迁移 GPU；未来可把连续窗口组成 batch，并按显存动态缩放 batch size。
- 结果可进一步转换为严格 COG，并接入 TiTiler/STAC。

13. 资产、瓦片、统计和溯源产物

13.1 影像检查与传感器补全

``inspect_raster`` 返回尺寸、CRS、bounds、nodata、波段描述、波长和 overview。GeoTIFF 自带描述和波长优先；GF01、LAD08/LAD09、SHB02 文件名前缀配置只在元数据缺失且波段数匹配时补全。

13.2 动态瓦片

``/api/tiles/{z}/{x}/{y}.png`` 把 Web Mercator 瓦片范围转换到影像 CRS，只读取当前瓦片覆盖区域；单波段结果使用拉伸色带，多波段源影像选择显示波段生成 RGB。缓存键包含文件修改时间，文件更新后自动失效。

13.3 路径安全

对象键和本地路径最终必须解析到受允许的数据目录。接口不直接信任用户输入的任意文件系统路径。这是防止目录穿越和读取服务器敏感文件的基础边界。

14. 异步任务、Celery、Redis 与取消

14.1 JobRecord 状态模型

- accepted：已接收、尚未开始。
- running：正在处理，包含 current/total/message。
- successful：结果可从 record.result 或 results 接口读取。
- failed：保存错误文本和结束时间。

- dismissed：任务被取消或撤销。

14.2 本地与部署模式统一

开发模式使用 `ThreadPoolExecutor`，便于不启动 Redis/Celery 也能演示异步接口；部署模式调用 ``celery_app.send_task``。两者共享 ``RasterTask``、``JobRecord`` 和 ``RasterPipeline``，减少环境差异。

14.3 优先级

请求 priority 为 1~5。Celery 配置五级队列并启用优先级参数；本地线程池不具备严格队列优先级语义，因此答辩时应说明“接口与部署队列支持优先级，本地轻量模式主要用于开发”。

15. 智能分析 Agent 的实现与安全边界

15.1 Agent 不是直接执行器

```
用户问题
→ 创建/恢复会话
→ 注册表与外部知识 RAG
→ 可选网络检索
→ 可选 LLM 意图分类
→ 确定性规则选择候选指数
→ 检查当前波段是否可执行
→ 生成执行单（awaiting_confirmation）
→ 用户编辑并确认
→ 后端二次校验
→ 提交 JobManager
```

15.2 推荐依据

- IntentRule：长势、稀疏植被、叶绿素、胁迫、变化等稳定任务模板。
- INDEX_REGISTRY：公式、波段、标签、限制，是推荐事实源。
- Knowledge Store：内存/PostgreSQL 外部知识，按词项评分召回。
- Web Search：可选增强，失败时降级本地知识。
- LLM：可选意图分类，不直接生成可执行 Python 或存储路径。

15.3 三道执行安全门

- 33. 计划阶段：缺少波段的指数标记 executable=false，不进入 selectedIndices。
- 34. 确认阶段：用户提交的 indices 必须是允许集合子集，否则 422。
- 35. 任务阶段：再次打开源影像，检查波段号在 1~source.count 范围内。

15.4 自定义指数安全

表达式先由 `SafeExpressionValidator` 检查 AST，只允许算术、白名单函数和已声明波段；禁止属性访问、下标逃逸、导入和任意函数调用。随后用小型探针数组试算，确认输出形状、范围和有限性，才加入注册表。

16. Vue 前端各层实现

文件/组件	主要职责	关键状态/交互
App.vue	根编排与轮询	系统刷新、任务刷新、面板切换
workspace.ts	单一状态源	asset、bands、indices、jobs、result、panel
usePlatformApi.ts	REST/SSE 客户端	JSON、XHR 上传进度、SSE 帧解析
AssetToolbar.vue	上传和波段映射	拖拽、overview、逻辑波段、批量提交
IndexCatalog.vue	指数目录	分类、搜索、公式 token 与分式排版
MapWorkspace.vue	地图图层	天地图、源 TIF、结果瓦片、定位、对比
JobProgressPanel.vue	任务中心	状态、ETA、吞吐、产品和取消
StatisticsDashboard.vue	统计图表	ECharts 生命周期和产品切换
AgentDrawer.vue	Agent 交互	会话、SSE 思考、执行单、确认和解释

16.1 为什么 Pinia 是单一状态源

上传组件、地图、任务面板、统计和 Agent 都需要共享当前资产、波段映射和活动结果。若各组件分别保存，很容易出现地图展示 A 结果、统计却读取 B 结果。`workspace.ts` 集中维护并用 computed 派生活动产品。

16.2 SSE 解析

`requestStream` 从 ReadableStream 累积文本，按空行拆分 SSE frame，`parseSseFrame` 读取 event/data。AgentDrawer 根据 thought/status/plan/job/result/error/done 更新不同 UI 状态，并用队列控制思考动画节奏。

16.3 地图稳定性设计

- `mapWhenStyleReady` 防止 style 尚未加载时调用 addLayer。
- 源影像使用 preview、范围和动态瓦片多级兜底，避免上传后出现空白。
- 图层顺序固定为范围、源影像、结果，避免分析层被底图覆盖。
- 自动定位只对新资产触发，用户手动浏览后不反复抢夺视角。
- 瓦片 URL 使用 objectKey，避免浏览器直接访问服务器本地路径。

17. 部署、配置、测试与常见问题

17.1 Docker Compose 组件

- API/OGC 服务实例：承载 HTTP 和 Process 能力。
- Celery Worker：执行长时间栅格任务。
- Redis：broker 与结果后端。
- MinIO：影像和结果工件。
- Nacos：服务注册与发现。
- Nacos Bridge：把健康实例转换成 Traefik 动态配置。
- Traefik：统一网关和负载均衡。

- Vue/Nginx：前端静态资源和反向代理。

17.2 质量检查命令

```
cd backend
D:\miniconda\envs\giskeshe\python.exe -m ruff check .
D:\miniconda\envs\giskeshe\python.exe -m pytest -q

cd ../frontend
npm run build
```

17.3 测试覆盖地图

测试文件	覆盖重点
test_indices.py	注册表数量、NDVI 手算、有限值、Joblib/Torch 一致性
test_raster_pipeline.py	分块输出尺寸、CRS、transform 和产品结构
test_assets.py	overview 构建复用、传感器配置、元数据保护
test_api.py	健康、Process、上传、瓦片、异步、Agent 和能力接口
test_agent.py	推荐、缺波段阻断、确认、RAG、自定义指数
test_advanced_analysis.py	AST 白名单、变化检测、GeoJSON 统计

17.4 常见问题

现象	优先检查	原因/处理
后端启动失败	是否使用 giskeshe Python	Rasterio/GDAL/Pydantic 版本必须来自固定环境
地图无底图	VITE_TIANDITU_TOKEN	无令牌时使用兜底底图或检查网络
指数不可执行	requiredBands 与 bands	逻辑波段缺失或物理波段号越界
Torch 实际变 Joblib	fallbackReasons	CUDA 不可用、导入失败或运行失败
任务一直 accepted	Redis/Worker/eager	部署队列未启动或本地模式配置不一致
结果瓦片空白	objectKey、bounds、CRS、overview	路径、投影或数据范围问题

现象	优先检查	原因/处理
Swagger 上传报 422	文件类型和 multipart	只接受可保存并由 Rasterio 打开的影像

18. 汇报演示脚本与高频答辩问答

18.1 8 分钟演示顺序

36. 用 30 秒说明统一注册表、分块流水线、三引擎、OGC 服务和 Vue 工作台。
37. 打开 `docs`，展示 `/api/indices/ndvi` 与 `/api/system/capabilities`。
38. 上传或选择 GeoTIFF，解释逻辑波段映射。
39. 选择 NDVI/EVI/GNDVI，提交异步任务，展示 jobID 和窗口进度。
40. 切换地图结果和统计图，指出实际引擎、统计和 manifest。
41. 在 Agent 输入“找长势不好的区域”，展示推荐、缺波段阻断和确认按钮。
42. 打开 `indices.py`、`raster\_pipeline.py` 和 `routes.py`，对应讲公式、分块和接口。
43. 最后说明 CUDA/容器等环境边界与后续优化。

18.2 高频问题与回答

问：为什么不用一个函数写完？

答：公式、I/O、协议、任务和 UI 的变化原因不同。分层后每层可独立测试，新增接口或引擎不需要复制公式。

问：35 个指数为什么只维护一份？

答：IndexDefinition 把公式表达式参数化为 `xp`；NumPy 和 Torch 只替换数组 API。

问：波段映射为什么用 red/nir 而不是 3/4？

答：物理波段号随传感器变化，逻辑波段让公式保持传感器无关。

问：分块后结果会不会有拼接缝？

答：指数是逐像元运算，不依赖邻域；Window 写回原位置，不会产生边界缝。

问：块越小越好吗？

答：不是。太小会增加 Rasterio 调用、调度和写入次数；太大增加内存和 GPU 显存。

问：为什么默认 1024？

答：它在 I/O 次数和内存之间较均衡，并满足 GeoTIFF 分块的 16 倍数要求。

问：为什么掩膜做逻辑或？

答：任一必需波段无效，该像元的公式输入就不完整，结果应统一为 nodata。

问：如何保证三引擎一致？

答：同一 expression、统一 safe_divide/sanitize_result，并用固定数组比较。

问：Torch 失败怎么办？

答：TorchEngine 捕获导入、CUDA 和运行异常，回退 Joblib，并记录 fallbackReason。

问：为什么 Joblib 用线程？

答：NumPy 核心运算通常释放 GIL，线程可共享窗口数组，避免进程复制大数组。

问：同步和异步是否两套算法？

答：不是。区别只在调用 JobManager 的 execute_sync 或 submit，最终都进入 RasterPipeline。

问：如何取消任务？

答：JobManager 设置取消状态/撤销 Celery；流水线每个窗口前检查取消回调并在 finally 关闭 writer。

问：pygeoapi 和 FastAPI 有什么区别？

答：pygeoapi 是标准 OGC 框架入口；FastAPI 是综合平台入口，包含上传、Agent、瓦片等扩展。

问：为什么只写一个 Processor？

答：请求 index 动态查注册表，避免 35 个类和 YAML 重复。

问：接口文档在哪里？

答：FastAPI 为 /docs、/redoc、/openapi.json；pygeoapi 为 Landing Page 和 /openapi.json。

问：如何防止自定义公式执行恶意代码？

答：使用 AST 白名单，只允许指定名称、算术节点和函数，再用探针数组试算。

问：Agent 会不会自己执行？

答：不会。计划状态为 `awaiting_confirmation`，确认接口还会二次验证可执行指数和波段。

问：RAG 的知识从哪里来？

答：内置注册表、用户导入知识、PostgreSQL 持久知识和可选网络检索。

问：为什么需要 manifest？

答：记录输入哈希、参数、引擎、回退、环境和输出，使结果可复现、可审计。

问：overview 会改变计算精度吗？

答：不会。overview 只是主波段的低分辨率浏览层，计算和下载仍用全分辨率数据。

问：统计为什么排除 -9999？

答：它是统一输出 `nodata`，不代表真实指数值；纳入会严重扭曲均值和直方图。

问：MinIO 不可用怎么办？

答：核心结果先写本地；上传适配返回空对象键，接口仍保留本地路径供开发。

问：Nacos 为什么还需要 Bridge？

答：Traefik 没有直接使用本项目 Nacos 信息的原生链路，Bridge 把实例转换成 File Provider 配置。

问：当前最大性能瓶颈？

答：通常是磁盘读取、结果写出和全量统计，不一定是公式本身。

问：下一步最值得优化什么？

答：流式窗口生成、在线统计、GPU batch、严格 COG/TiTiler、基准反馈自动规划器。

问：如何证明没有整幅读入？

答：核心计算只在窗口循环中调用 `source.read(window=window)`；整幅读取只在可选统计阶段读取单个结果波段。

问：统计阶段仍可能占内存吗？

答：是。当前 `dataset.read(1)` 会读单个结果全幅；这是已明确的后续在线统计优化点。

问：为什么前端不计算 NDVI？

答：浏览器不适合处理大 GeoTIFF；后端统一保证投影、nodata、分块和可复现性。

问：为什么用 Pinia？

答：地图、任务、Agent 和统计共享状态，单一 store 防止多个组件状态不一致。

问：如何处理传感器差异？

答：优先读取影像元数据；必要时按文件名前缀和波段数补全，再由用户确认逻辑映射。

18.3 最后一分钟总结模板

可直接口述：这个项目的核心不是单独写出 NDVI，而是把 35 个指数抽象成统一注册表，再通过分块流水线、多引擎、标准接口和可视化工作台形成完整系统。公式层与框架解耦，FastAPI、Celery 和 pygeoapi 最终复用同一套 RasterPipeline；Agent 只负责规划并受确认机制约束。当前实现已具备完整演示链路，后续重点是目标硬件基准、在线统计和更标准的 COG 服务。

附录 A. FastAPI 全接口表

当前从 `backend/app/api/routes.py` 自动提取 30 个接口。

方法	路径	处理函数	职责
GET	/api/indices	list_indices	列出内置和动态指数，并支持分类与波段筛选。
GET	/api/indices/{index_id}	index_detail	执行 index_detail 对应的领域操作并返回结构化结果。
GET	/processes	list_processes	执行 list_processes 对应的领域操作并返回结构化结果。
GET	/processes/{process_id}	describe_process	执行 describe_process 对应的领域操作并返回结构化结果。
POST	/processes/{process_id}/execution	execute_process	根据 Prefer 头选择同步结果或异步任务响应。
GET	/jobs	list_jobs	执行 list_jobs 对应的领域操作并返回结构化结果。
GET	/jobs/{job_id}	get_job	执行 get_job 对应的领域操作并返回结构化结果。
GET	/jobs/{job_id}/results	get_job_results	执行 get_job_results 对应的领域操作并返回结构化结果。
DELETE	/jobs/{job_id}	cancel_job	执行 cancel_job 对应的领域操作并返回结构化结果。
POST	/api/assets/inspect	inspect_asset	执行 inspect_asset 对应的领域操作并返回结构化结果。
POST	/api/assets/upload	upload_asset	执行 upload_asset 对应的领域操作并返回结构化结果。
POST	/api/assets/upload-url	upload_url	执行 upload_url 对应的领域操作并返回结构化结果。
GET	/api/tiles/{z}/{x}/{y}.png	geotiff_tile	执行 geotiff_tile 对应的领域操作并返回结构化结果。
POST	/api/agent/plan	create_agent_plan	执行 create_agent_plan 对应的领域操作并返回结构化结果。
POST	/api/agent/plan/stream	stream_agent_plan	通过 SSE 分阶段返回状态、思考摘要和最终计划。
POST	/api/agent/chat	chat_with_agent	执行 chat_with_agent 对应的领域操作并返回结构化结果。

方法	路径	处理函数	职责
POST	/api/agent/plans/{plan_id}/confirm	confirm_agent_plan	执行 confirm_agent_plan 对应的领域操作并返回结构化结果。
POST	/api/agent/plans/{plan_id}/confirm/stream	stream_confirm_agent_plan	通过 SSE 提交已确认计划并推送任务直到终态。
POST	/api/agent/interpret-results	interpret_agent_results	执行 interpret_agent_results 对应的领域操作并返回结构化结果。
GET	/api/agent/sessions/{session_id}/events	get_agent_session_events	执行 get_agent_session_events 对应的领域操作并返回结构化结果。
POST	/api/agent/knowledge	import_agent_knowledge	执行 import_agent_knowledge 对应的领域操作并返回结构化结果。
POST	/api/indices/custom	create_custom_index	执行 create_custom_index 对应的领域操作并返回结构化结果。
GET	/api/recipes	list_recipes	执行 list_recipes 对应的领域操作并返回结构化结果。
POST	/api/recipes	create_recipe	执行 create_recipe 对应的领域操作并返回结构化结果。
POST	/api/formulas/validate	validate_formula	执行 validate_formula 对应的领域操作并返回结构化结果。
POST	/api/analysis/change	change_detection	执行 change_detection 对应的领域操作并返回结构化结果。
POST	/api/analysis/zonal-statistics	zonal_statistics	执行 zonal_statistics 对应的领域操作并返回结构化结果。
GET	/api/benchmarks/engines	engine_benchmarks	执行 engine_benchmarks 对应的领域操作并返回结构化结果。
GET	/api/system/capabilities	capabilities	执行 capabilities 对应的领域操作并返回结构化结果。
GET	/api/system/taskbook-coverage	taskbook_coverage	执行 taskbook_coverage 对应的领域操作并返回结构化结果。

附录 B. 35 个植被指数注册表

当前统一注册表共 35 个内置指数。公式文本用于展示，实际计算使用 expression。

ID	中文名	公式	所需逻辑波段	分类
NDVI	归一化植被指数	$(\text{NIR}-\text{Red})/(\text{NIR}+\text{Red})$	nir, red	vegetation, biomass
GNDVI	绿色归一化植被指数	$(\text{NIR}-\text{Green})/(\text{NIR}+\text{Green})$	nir, green	chlorophyll
BNDVI	蓝色归一化植被指数	$(\text{NIR}-\text{Blue})/(\text{NIR}+\text{Blue})$	nir, blue	vegetation, visible
NDRE	归一化红边指数	$(\text{NIR}-\text{RedEdge})/(\text{NIR}+\text{RedEdge})$	nir, red_edge	red-edge, chlorophyll
NORMB	标准化蓝指数	$\text{Blue}/(\text{Red}+\text{Green}+\text{Blue})$	blue, red, green	visible, color-normalized
RVI	比值植被指数	NIR/Red	nir, red	vegetation
GR	绿红比值指数	Green/Red	green, red	visible
DVI	差值植被指数	$\text{NIR}-\text{Red}$	nir, red	vegetation
MSR	改进简单比值植被指数	$(\text{NIR}/\text{Red}-1)/\text{sqrt}(\text{NIR}/\text{Red}+1)$	nir, red	vegetation, biomass
RDVI	再归一化植被指数	$(\text{NIR}-\text{Red})/\text{sqrt}(\text{NIR}+\text{Red})$	nir, red	vegetation, biomass
SAVI	土壤调节植被指数	$(1+L)*(\text{NIR}-\text{Red})/(\text{NIR}+\text{Red}+L)$	nir, red	soil-adjusted
OSAVI	优化土壤调节植被指数	$1.16*(\text{NIR}-\text{Red})/(\text{NIR}+\text{Red}+0.16)$	nir, red	soil-adjusted
MSAVI	修正土壤调节植被指数	$(2*\text{NIR}+1-\text{sqrt}((2*\text{NIR}+1)^2-8*(\text{NIR}-\text{Red}))))/2$	nir, red	soil-adjusted
EVI	增强植被指数	$G*(\text{NIR}-\text{Red})/(\text{NIR}+\text{C1}*\text{Red}-\text{C2}*\text{Blue}+L)$	nir, red, blue	vegetation, atmosphere-resistant
EVI2	双波段增强植被指数	$2.5*(\text{NIR}-\text{Red})/(\text{NIR}+2.4*\text{Red}+1)$	nir, red	vegetation
ARVI	抗大气植被指数	$(\text{NIR}-(2*\text{Red}-\text{Blue})) / (\text{NIR}+(2*\text{Red}-\text{Blue}))$	nir, red, blue	atmosphere-resistant
VARI	可见光抗大气指数	$(\text{Green}-\text{Red})/(\text{Green}+\text{Red}-\text{Blue})$	green, red, blue	visible
GLI	绿叶指数	$(2*\text{Green}-\text{Red}-\text{Blue})/(2*\text{Green}+\text{Red}+\text{Blue})$	green, red, blue	visible

ID	中文名	公式	所需逻辑波段	分类
NGRDI	归一化绿红差异指数	$(\text{Green}-\text{Red})/(\text{Green}+\text{Red})$	green, red	visible
EXG	过绿指数	$2*\text{Green}-\text{Red}-\text{Blue}$	green, red, blue	visible
TVI	变换植被指数	$\text{sqrt}(\text{max}(\text{NDVI}+0.5,0))$	nir, red	vegetation
CTVI	修正变换植被指数	$\text{sign}(\text{NDVI}+0.5)*\text{sqrt}(\text{abs}(\text{NDVI}+0.5))$	nir, red	vegetation
IPVI	红外百分比植被指数	$\text{NIR}/(\text{NIR}+\text{Red})$	nir, red	vegetation
GCI	绿色叶绿素指数	$\text{NIR}/\text{Green}-1$	nir, green	chlorophyll
RECI	红边叶绿素指数	$\text{NIR}/\text{RedEdge}-1$	nir, red_edge	red-edge, chlorophyll
MCARI	修正叶绿素吸收比值指数	$((\text{RE}-\text{Red})-0.2*(\text{RE}-\text{Green}))*(\text{RE}/\text{Red})$	red_edge, red, green	chlorophyll, red-edge
TCARI	转换叶绿素吸收比值指数	$3*((\text{RE}-\text{Red})-0.2*(\text{RE}-\text{Green}))*(\text{RE}/\text{Red})$	red_edge, red, green	chlorophyll, red-edge
MTCI	陆地叶绿素指数	$(\text{NIR}-\text{RE})/(\text{RE}-\text{Red})$	nir, red_edge, red	chlorophyll, red-edge
PSRI	植物衰老反射指数	$(\text{Red}-\text{Green})/\text{RE}$	red, green, red_edge	senescence
SIPI	结构不敏感色素指数	$(\text{NIR}-\text{Blue})/(\text{NIR}-\text{Red})$	nir, blue, red	pigment
NDMI	归一化水分指数	$(\text{NIR}-\text{SWIR1})/(\text{NIR}+\text{SWIR1})$	nir, swir1	water
NDWI	归一化水体指数	$(\text{Green}-\text{NIR})/(\text{Green}+\text{NIR})$	green, nir	water
MSI	水分胁迫指数	$\text{SWIR1}/\text{NIR}$	swir1, nir	water
NBR	归一化燃烧比	$(\text{NIR}-\text{SWIR2})/(\text{NIR}+\text{SWIR2})$	nir, swir2	disturbance
BSI	裸土指数	$((\text{SWIR1}+\text{Red})-(\text{NIR}+\text{Blue}))/((\text{SWIR1}+\text{Red})+(\text{NIR}+\text{Blue}))$	swir1, red, nir, blue	soil

参考与事实来源

- 项目任务书：docx/植被指数提取算法封装与 Web 服务实现实习任务书.docx。
- 当前源码：backend/app/、backend/tests/、frontend/src/、infra/、compose.yml。
- pygeoapi 官方文档：OGC API - Processes、插件开发、运行方式与 OpenAPI 页面。
- 接口和指数附录由本生成脚本从当前源码自动提取，避免手工清单漂移。

时效说明：本手册生成日期见封面。若后续新增接口、指数或文件，请重新运行 build_handbook.py 并重新做 DOCX 渲染检查。
